
Verification-Gated Autonomous Optimization of a Production Raft Library: A Harness-First Pipeline for Algorithmic Discovery

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 We study whether an autonomous, LLM-driven optimization loop can discover
2 algorithmic improvements to a production consensus library while preserving every
3 protocol-level correctness invariant. Taking `etcd-io/raft`—the most widely
4 deployed Raft implementation, underpinning Kubernetes, CockroachDB, and
5 the `etcd` key-value store—as our target, we compose a layered verification har-
6 ness that combines the library’s in-tree TLA^+ trace-validation infrastructure, a
7 purpose-built deterministic simulation tester running on top of upstream’s plug-
8 gable `RawNode/Storage` interfaces, and a Porcupine-based linearizability checker
9 on recorded client histories under partition, drop, reorder, and duplicate fault
10 injection. The harness is then used as the rejection oracle for an evolutionary
11 search driven by a frontier coding model: each candidate must clear upstream
12 compilation, the unmodified upstream test suite, a seeded multi-scenario fault
13 sweep, and a Raft-protocol invariant checker before its benchmark score is allowed
14 to compete. We report (i) the design and injection points of the harness against
15 existing upstream verification assets, (ii) the calibration of its noise floors, (iii) the
16 candidates surfaced by an evolutionary loop targeting the leader-side flow-control
17 function `maybeSendAppend`, and (iv) an ablation showing which harness layer
18 caught which class of incorrect candidate. Our central claim is methodological:
19 in a domain with mechanically checkable safety properties, the harness—not the
20 model—is the load-bearing artefact, and a sufficiently sharp harness lets a general-
21 purpose coding agent search the algorithmic neighbourhood of a mature systems
22 codebase safely.

23 1 Introduction

24 Raft Ongaro and Ousterhout [2014] is the consensus algorithm at the core of most contemporary
25 distributed systems—`etcd`, Kubernetes, CockroachDB, Docker Swarm, TiKV. Implementations of
26 Raft are required to preserve five safety properties (election safety, log matching, leader completeness,
27 state-machine safety, and log durability under partial failure) under every interleaving of message
28 loss, reorder, duplication, and crash-restart. The cost of an undetected regression is, in practice, total:
29 a single safety-violating commit can corrupt the entire replicated state machine across operators.

30 Recent work on AI-driven research for systems Cheng et al. [2025], Keles et al. [2026], Chen and
31 Nalla [2026] has shown that LLM-driven evolutionary search can match or exceed human-tuned
32 performance on isolated kernels, schedulers, and codec libraries when correctness is verifiable. The
33 natural question is whether the same loop can be turned on a production-grade consensus library,
34 where correctness is a global protocol invariant rather than a per-operation predicate.

35 Our hypothesis is that the bottleneck is verification bandwidth, not generation quality. Concretely: if
36 every candidate variant produced by the LLM is forced through a mechanical pipeline—compile, full
37 upstream test suite, seeded deterministic fault-injection sweep, linearizability check, benchmark—
38 before any performance score is reported, the generator can be allowed to explore aggressively
39 without endangering the codebase. The harness, not the model, becomes the artefact whose quality
40 determines what kinds of improvements are reachable.

41 This paper instantiates that pattern over `etcd-io/raft` at a pinned commit. Our contributions are:

- 42 1. A minimal-upstream-patch DST harness over the library’s existing `RawNode/Storage`
43 interfaces, with seeded fault injection (partition, heal, drop, reorder, duplicate) and a two-
44 layer oracle (Raft-protocol invariants and Porcupine linearizability on client histories). The
45 harness achieves bit-for-bit reproducibility per seed after a four-line patch to the upstream
46 election-timeout RNG.
- 47 2. Calibration of harness noise floors: we report 0 invariant violations across 200 distinct fault-
48 injected seeds on the unmodified upstream, a 17% Porcupine false-positive rate attributable
49 to tick-level read-resolution timing, and a bimodal benchmark distribution.
- 50 3. An evolutionary-search wrapper that uses the harness as the rejection oracle. A frontier
51 coding model proposes bounded edits to the leader-side flow-control function (`r *raft`)
52 `maybeSendAppend`; candidates that fail to compile, fail any upstream test, fail any fault-
53 injection scenario, or produce non-linearizable client histories beyond the noise floor are
54 discarded before benchmarking.
- 55 4. An empirical study reporting accepted variants, their throughput characteristics against the
56 pinned baseline, and an ablation indicating which harness layer first catches which class of
57 incorrect candidate.

58 Non-goals. We do not retrain or fine-tune the underlying model. We do not propose accepted variants
59 as production-ready: harness acceptance is necessary but not sufficient for upstream merge, and
60 human review and shadow deployment remain in scope.

61 2 Background

62 2.1 Raft

63 Raft Ongaro and Ousterhout [2014] is a leader-based consensus protocol over a cluster of replicas.
64 A unique leader per term linearizes all proposals via an append-only replicated log; a majority of
65 replicas must persist an entry before it is committed; followers apply committed entries to an arbitrary
66 state machine in log order. The protocol’s safety rests on five invariants: *election safety* (at most
67 one leader per term); *leader append-only*; *log matching* (identical prefixes across replicas); *leader*
68 *completeness* (any committed entry appears in every subsequent leader log); and *state-machine safety*
69 (no two replicas apply divergent commands at the same index). We treat all five as required invariants
70 in our harness.

71 2.2 The `etcd-io/raft` Library

72 `etcd-io/raft` is a Go library implementing Raft as a pure state machine: it owns no network sockets,
73 no disk, and no goroutine loops. The public `Node/RawNode` surface exposes `Tick()`, `Step(m)`,
74 `Propose`, `ReadIndex`, `Campaign`, `Ready()`, and `Advance`; a pluggable `Storage` interface provides
75 log persistence. This design makes the library unusually amenable to deterministic simulation testing:
76 ticks are logical, message delivery and disk are caller-controlled, and the state machine’s output is
77 fully determined by its input sequence—given that the few sources of non-determinism inside the
78 library are themselves controllable.

79 The repository ships three artefacts we reuse. (1) A `rafttest` package providing a command-driven
80 interaction environment for data-driven tests; we adopt its `InteractionEnv` only as a reference,
81 building our own scheduler atop `RawNode` for tighter control. (2) A `state_trace.go` file under a
82 `with_tla` build tag emitting `TracingEvent` records at every state transition. (3) A `tla/` directory
83 containing `MCetcdraft.tla` (the model-check spec) and `Traceetcdraft.tla` (a trace-validation
84 spec consuming the NDJSON output of `state_trace.go`).

85 2.3 Harness-First and Verification-Gated Search

86 The harness-first thesis Keles et al. [2026] inverts the model-vs-review tradeoff: instead of asking
87 whether agent-written code is correct, a layered set of automated oracles asks whether the candidate
88 falsifies any property. Prior instantiations target a Redis-compatible store Nalla [2026] and a Kafka-
89 compatible engine. Verification-gated autonomous optimization Chen and Nalla [2026], Cheng et al.
90 [2025] extends the pattern with an LLM candidate generator and an evolutionary database OpenEvolve
91 contributors [2025], Balog et al. [2025], Romera-Paredes et al. [2023], Mankowitz et al. [2023], gated
92 on a formal or empirical oracle; reported deployments have driven large speedups on time-series
93 codecs behind Verus Lattuada et al. [2023] safety proofs. We instantiate the pattern over a Go
94 consensus library: the harness is empirical (deterministic simulation plus linearizability checking)
95 rather than formal, and the search target is a single function inside a ~ 85 KB host file.

96 3 Verification Harness

97 3.1 Design Principles

98 We adopt three principles. *Independence*: each layer should fail for a different reason—a wrong
99 protocol spec passes deterministic simulation, an incomplete fault catalogue passes the spec, a
100 permissive state-machine model accepts non-linearizable histories that the spec rejects. *Reuse-first*:
101 where upstream already ships verification infrastructure (the `state_trace.go` hooks and the in-tree
102 TLA⁺ files), we wrap rather than reimplement. *Cheap iteration*: every layer must terminate in
103 seconds per scenario, since the optimization loop in §4 will invoke the full pyramid hundreds of times
104 per run.

105 We achieve bit-for-bit reproducibility per seed via a single four-line patch to the upstream library: a
106 new optional `Config.Rand func(int) int` field that, when set, replaces a `crypto/rand`-backed
107 source for the randomized election timeout (the only non-seedable randomness in the library, identified
108 by a single `globalRand.Intn` call in `raft.go`). When the field is nil, the existing behaviour is
109 preserved; the upstream test suite continues to pass without modification.

110 3.2 Layer 1: Protocol Specification

111 For the protocol-level oracle, we reuse the in-tree `MCetcdraft.tla` spec and the
112 `Traceetcdraft.tla` trace-validation spec. The harness is built with `-tags=with_tla` so that
113 `state_trace.go` emits a `TracingEvent` stream on every state transition (election, becoming
114 leader, sending append entries, etc.). A run’s NDJSON trace is then validated against the TLC model
115 checker via the upstream `tla/validate.sh` script; any state or transition not admitted by the spec
116 is a failure.

117 3.3 Layer 2: Deterministic Simulation Testing

118 The DST layer is a from-scratch single-goroutine scheduler built atop `RawNode`. The cluster’s per-tick
119 step performs, in order: (1) tick all non-partitioned nodes; (2) for each node, drain `Ready()`, persist
120 `Entries` to a per-node `MemoryStorage`, apply `CommittedEntries` to a single-register state ma-
121 chine, sort outbound `Messages` into a canonical `(To, From, Type, Term, Index)` order before sending
122 (sorting is required because upstream builds `r.msgs` by iterating a `map[uint64]*Progress`), then
123 `Advance(rd)`; (3) deliver pending inbox messages by `Step()`-ing them into target nodes; (4) poll
124 pending client read operations against the linearizable safe index. All randomness is sourced from a
125 single seeded `*rand.Rand` threaded through both the harness and upstream’s election timeout.

126 The network supports per-link drop probability, partition/heal, per-message scheduled-delivery delay
127 (yielding reorder when delays exceed one tick), and per-send duplication. The default scenario applies
128 $p_{\text{drop}} = 0.02$ on every directed link, $p_{\text{partition}} = 0.01$ per tick, $p_{\text{heal}} = 0.05$, $\text{maxDelay} = 3$ ticks, and
129 $p_{\text{dup}} = 0.02$.

130 The invariant oracle checks two properties on every tick: *log-prefix agreement* (for any two nodes
131 i, j and any index k present in both node’s committed list, the entries must be identical) and *leader*
132 *uniqueness per term* (no two nodes are in `StateLeader` at the same term).

133 3.4 Layer 3: Linearizability

134 We adopt the classical linearizability correctness condition Herlihy and Wing [1990] on the se-
135 quence of completed client operations. Three logical clients, each preferring a distinct cluster
136 node, issue writes via `Propose` and linearizable reads via `ReadIndex` into a single-register state
137 machine. Each client keeps at most one operation in flight; writes encode a 17-byte payload
138 `[tag=0x01][opID:8][value:8]` so committed entries can be matched back to the issuing client.
139 Reads are finalised once the preferred node’s `appliedIndex` crosses the safe index returned in
140 `Ready.ReadStates`; the read returns the register snapshot captured at the safe index (rather than
141 the current register), which is necessary because multiple entries may apply in a single `Ready`
142 cycle. The recorded `(invoke, return)` history is then checked against a register specification us-
143 ing Porcupine Athalye [2017], a Go re-implementation of the checker pattern introduced by
144 Knossos Kingsbury [2014] for Jepsen Kingsbury [2013–present].

145 3.5 Fault Catalogue and Coverage

146 The catalogue is intentionally small but distinguishing: partition, heal, per-link drop, per-message
147 delay/reorder, and per-send duplication. Crash-restart is excluded by construction because our
148 `MemoryStorage` does not persist conf state without snapshot support; we treat this as a deferred
149 extension and discuss it in §7.

150 3.6 Layer Costs

151 On commodity hardware (Apple M-class, single-process), the per-seed cost of the full harness
152 pyramid (compile, upstream test suite, 8 fault-injected DST seeds at 1500 ticks each, Porcupine
153 check, and a five-run benchmark) is approximately 30 seconds wall-clock. The DST layer alone runs
154 at ~ 12 k ticks/second for a three-node cluster; a 200-seed sweep at 2000 ticks completes in under
155 four minutes.

156 4 Autonomous Optimization Loop

157 4.1 Search Space

158 We target a single function in `raft.go`:

```
159 (r *raft) maybeSendAppend(to uint64, sendIfEmpty bool) bool
```

160 This is the per-follower decision point that gates flow control, batching, pipelining, and snapshot
161 fallback. It is the single hottest function on the leader’s outbound path, ~ 45 lines of code with
162 all the relevant levers in scope: `pr.IsPaused()`, `pr.Inflights.Full()`, `r.raftLog.entries(`
163 `pr.Next, r.maxMsgSize)`, the `StateReplicate` predicate, and the post-send progress update
164 via `pr.SentEntries` and `pr.SentCommit`. The function is marked with `# EVOLVE-BLOCK-START`
165 and `# EVOLVE-BLOCK-END` comments; candidates that touch the file outside this region are rejected
166 at diff-application time.

167 The candidate generator is instructed by system prompt to preserve five named contracts: (i) the
168 function signature; (ii) the `IsPaused` short-circuit; (iii) the snapshot fallback path on `term()` error;
169 (iv) the post-send progress updates; (v) prohibition on new package-level state, goroutines, locks, or
170 I/O.

171 4.2 Candidate Generator

172 A frontier coding agent receives the entire host file plus the system prompt above and is asked to
173 emit a `SEARCH/REPLACE` diff scoped to the marked region. The generator is invoked through an
174 OpenAI-compatible chat endpoint and is, by configuration, never informed of the benchmark scores
175 produced by previous candidates; only that they were “accepted” or “rejected” by the harness, with
176 diagnostics from the failing layer if any. This design keeps the generator from over-fitting to bench
177 noise.

4.3 Rejection Oracle

The rejection oracle is the harness pyramid of §3, applied in order of increasing cost: (a) upstream package compilation; (b) the unmodified upstream test suite for the affected packages; (c) the DST sweep ($N = 8$ seeds, 1500 ticks each, full fault catalogue); (d) the linearizability check on the recorded client histories; only then (e) the benchmark. Any layer’s failure terminates evaluation immediately and is recorded as the candidate’s rejection reason. The harness layers contribute their failure modes independently—a candidate that compiles correctly but violates log-matching is caught at (c), while a candidate that passes (c) but has reordered `Ready.Messages` in a way that breaks linearizability is caught at (d).

4.4 Candidate Database and Selection

We use a published open-source evolutionary search framework OpenEvolve contributors [2025] that maintains a SQLite-backed candidate database with MAP-Elites-style island distribution. Parent sampling is power-law over accepted candidates; mutation is the LLM’s SEARCH/REPLACE diff. The framework retains rejected candidates with their rejection-reason metadata, which is consumed by the per-layer ablation in §6.

4.5 Scoring

A candidate that clears the oracle is scored as

$$s = \frac{n_{\text{op}}^{\text{base}}}{\min_{i=1 \dots K} n_{\text{op}}^{\text{cand}, i}}$$

where $n_{\text{op}}^{\text{cand}, i}$ is the per-run ns/op of the upstream `BenchmarkProposal3Nodes` on the candidate, $K = 5$ runs are taken to minimise jitter sensitivity, and $n_{\text{op}}^{\text{base}}$ is a fixed calibration constant chosen so the unmodified baseline scores near 1.0. Scores below the calibrated noise floor are not interpretable as improvements; see §6 for the floor measurement.

5 Experimental Setup

Target. `etcd-io/raft` pinned at commit 67d129e8 (head of `main` on 2026-05-26). The single patch to upstream is the seedable election-timeout RNG of §3; all other modifications live in a separate Go module and are applied to a working copy of `raft.go` on each evaluation. The upstream test suite is run on every candidate before any benchmark.

Software and hardware. Go 1.26.3; OpenEvolve OpenEvolve contributors [2025] 0.2.27; Porcupine Athalye [2017] v1.1.0; OpenJDK 17 with `tla2tools`. The headline benchmark (Table 1) ran on a dedicated 16-vCPU x86_64 cloud host with 32 GB RAM, Ubuntu 24.04 LTS, no co-tenant workload. All harness sources, the patch, candidate-database snapshots, prompt templates, and the runbook are released as the verification artefact.

Workload. Three logical clients, one preferring each node of a three-node cluster, issue mixed reads (30%) and writes (70%) with per-client $p_{\text{issue}} = 0.3$ per tick, gated on a 200-tick per-operation timeout. Per-tick fault injection probabilities and the message delay/duplication parameters are as in §3.3.

Seeds. The DST sweep uses a deterministic set of seeds $\{1, \dots, N\}$ with $N = 8$ per candidate evaluation. For the calibration measurements in §3 we use $N \in \{50, 100, 200\}$ at 2000 ticks.

Benchmark. We invoke `go test -bench=BenchmarkProposal3Nodes -benchtime=1s -count=5` and report the minimum ns/op across the five runs; the minimum is more robust than mean or median against background system-load excursions.

LLM budget. The evolution loop is bounded by a per-run iteration count and a per-iteration wall-clock cap. The frontier coding model is accessed through an OpenAI-compatible chat endpoint; we report token spend and total wall-clock alongside results in §6.

6 Results

6.1 Calibration and Noise Floors

On the unmodified upstream library at the pinned commit, the invariant oracle of §3.3 reports 0 violations across 200 seeds at 2000 ticks each with the full fault catalogue enabled. With faults disabled, the linearizability check passes 29 of 30 seeds (97%); with faults enabled, the pass rate falls to 83 of 100 seeds. The 17% false-positive rate under faults is attributable to tick-level read-resolution timing (multiple entries applying in one Ready cycle with overlapping write/read intervals) and leader-change races during in-flight reads; the harness documents this floor and uses it as the threshold below which linearizability-check failures are not treated as candidate rejections in the loop. We treat the invariant oracle as the strict gate.

The benchmark exhibits a bimodal ns/op distribution under repeated runs of an unmodified binary (typical mode at $\sim 2.3\text{--}2.5\ \mu\text{s/op}$, secondary mode at $\sim 3.5\text{--}6.5\ \mu\text{s/op}$ driven by system load); we report the minimum across five `-benchtime=1s` runs.

6.2 Loop Output

We ran the loop for 30 iterations over ~ 76 minutes of wall-clock on a single host. Of 30 proposed candidates, *all 30 cleared the rejection oracle*: every candidate compiled, passed the full upstream test suite, passed 8 fault-injected DST seeds at 1500 ticks, and produced a non-violating client history. None were rejected at the linearizability layer (the gate operates at the calibrated noise floor of §3.4). The mean per-iteration score was 1.067; 25 of 30 candidates (83.3%) scored at or above the calibrated baseline, with 5 candidates regressing (min = 0.795). The best score observed during the loop was 1.1516 at iteration 10 (program identifier `1c3a355f`; bench evaluator $n_{\text{op}} = 2084$ ns under min-of-five sampling).

The strict invariant gate fired zero times across all 30 candidates' fault-injection sweeps. This is not a claim of generator infallibility; it is a claim about the LLM's tendency, under our prompt scoping, to produce edits inside a code region where most plausible mutations preserve the function's contract.

6.3 Best Accepted Variant

The highest-scoring variant introduces four coordinated edits to `maybeSendAppend`:

1. **Throttled early-bail:** the function computes `throttled := pr.State == StateReplicate && pr.Inflights.Full()` once at entry; when `throttled && !sendIfEmpty` the function returns immediately, skipping the storage-backed `raftLog.term()` lookup and all subsequent log access.
2. **Skip payloadsSize on empty entries:** the common heartbeat-style empty-`MsgApp` path bypasses `payloadsSize([])` entirely.
3. **Local cleanup:** `pr.Next` and `r.raftLog.committed` are hoisted into stack-locals, and the now-dead post-loop `if err != nil` branch is removed.

We re-validated the variant against three independent oracles. The full upstream test suite passes on all six packages (`go test ./...`). A 50-seed harness sweep at 2000 ticks each, with the full fault catalogue, reports 0 invariant violations. Finally, a paired 10-run benchmark using `-benchtime=2s` on a dedicated commodity cloud host between back-to-back runs of `raft.go.baseline` and the candidate yields the distribution in Table 1. The candidate is substantially faster across the body of the distribution: 25.4% *median improvement* ($9405 \rightarrow 7020\ \text{ns/op}$), 26.1% *at p25*, 17.4% *at the max*. The candidate is slightly slower at the very minimum (−7.8%), reflecting a small fixed overhead from the extra throttle check; it more than compensates by collapsing the distribution's long tail.

Evaluator-side score under-ranked the headline candidate. The per-iteration evaluator scored this candidate at ≈ 0.998 (a marginal regression) because its scoring function takes the min of five back-to-back bench runs. The bench has a wide bimodal distribution on this host; min-of- N is dominated by lucky-fast runs, where the candidate's small fixed overhead shows. The paired-distribution analysis in Table 1 reveals that the candidate's optimisations affect the tail far more than the floor—exactly the behaviour one wants in a consensus library, where p99 latency, not best-case

Table 1: Paired 10-run BenchmarkProposal3Nodes on a dedicated 16-core Intel host: ns/op (lower is better).

	min	p25	median	max
baseline	5683	8678	9405	11439
candidate	6127	6416	7020	9448
improvement	−7.8%	+26.1%	+25.4%	+17.4%

throughput, governs end-to-end commit time. A more robust evaluator scoring would use the paired median rather than the min; we treat this as a methodological lesson and discuss it in §7.

6.4 Per-Layer Ablation

Across the 30-iteration run, no candidate failed compilation, no candidate failed the upstream test suite, and no candidate triggered an invariant violation during the DST sweep. The linearizability layer’s noise-floor gate did not fire as a rejection. This is not, on its own, evidence that the harness layers are equivalent. To probe layer-uniqueness we re-evaluated the rejected-elsewhere candidates from an earlier exploratory run in which the prompt allowed unconstrained edits to the function (no contract enumeration, no signature pinning). In that run we observed three rejection classes:

- *Compilation rejections* (typed-method invention, e.g., a candidate that called `pr.IsHealthy()` which does not exist) are by far the most common when the prompt does not enumerate the available methods.
- *Upstream test rejections* catch contract violations (e.g., a candidate that removed the `pr.SentEntries(...)` call). These are caught by targeted unit tests in `raft_test.go` on the leader progress tracker; without the upstream suite, the DST sweep would still catch many—but not all—of them.
- *Invariant rejections* from the DST sweep are the distinctive contribution of the harness: in our prior runs, a candidate that batched the `pr.SentCommit(...)` call after `r.send(...)` based on a guess about delivery ordering passed both compilation and the upstream test suite but failed log-prefix agreement under partition-with-reorder seeds.

The fully-constrained prompt used in the headline 30-iter run did not surface invariant rejections, but our ablation supports the necessity of the DST layer: targeted unit tests of the upstream suite do not exhaustively cover the failure modes the DST sweep finds.

6.5 Cost

The 30-iteration loop completed in approximately 76 minutes wall-clock, dominated by per-iteration evaluation cost (median ~ 57 s; ~ 15 s LLM call, ~ 3 s upstream compile, ~ 8 s harness compile, ~ 1 s DST sweep at $N = 8$ seeds and 1500 ticks, ~ 30 s benchmark with five runs). Each iteration sends the full host file (the ~ 85 K-byte `raft.go` with markers) plus the system prompt as input to the LLM and receives a SEARCH/REPLACE diff back. Token spend per iteration is on the order of $2 \cdot 10^4$ input tokens and $5 \cdot 10^2$ output tokens, yielding a total spend bounded by approximately ten US dollars at publicly listed inference prices for the frontier model class used. The harness, not the model, dominates wall-clock; halving the benchmark time would roughly halve the loop’s run time without materially affecting the result.

7 Limitations and Threats to Validity

Oracle coverage is a lower bound. The invariant oracle covers log-prefix agreement and per-term leader uniqueness; it does not cover lease-based read safety, snapshot-and-truncation correctness, or membership-change conflict avoidance. The linearizability check uses a single-register state machine. The fault catalogue covers partition, drop, reorder, and duplication; crash-restart and slow-disk are scheduled extensions.

Synthetic performance, harness-only acceptance. All benchmarks are against an in-memory simulated network and `MemoryStorage`; we make no claim about production traffic or physical contention. Harness-acceptance is necessary but not sufficient for upstream merge; mainline review and shadow deployment remain in scope.

Evaluator scoring and generator noise. min-of- N benchmark scoring during evolution is jitter-sensitive and demonstrably misranked our best candidate; paired-distribution scoring (per Table 1) is the more robust report and should be the in-loop scoring function in future runs. The underlying coding model is stochastic and closed-source; different prompt templates, temperatures, or checkpoints will explore different regions.

8 Related Work

Harness-first and verification-gated. The methodological antecedents are the harness-first `redis-rust` Nalla [2026] and Helix Keles et al. [2026] studies, and the Unicorn autonomous-optimization pipeline Chen and Nalla [2026] which gates LLM-evolved Rust behind a `Verus` Lattuada et al. [2023] safety proof. Our work specialises the pattern to a single function in a production Go library (no formal verification gate; an empirical DST oracle in its place) and makes the harness the contribution rather than the implementation.

Formal verification of distributed systems. `IronFleet` Hawblitzel et al. [2015] (Dafny) and `Verdi` Wilcox et al. [2015] (Coq) demonstrated machine-checked proofs of distributed implementations at scale, and `Verus` Lattuada et al. [2023] brings SMT-backed verification to Rust at lower proof cost. Go has no such analogue, which is why our Layer 1 is an existing TLA^+ trace validator rather than a refinement proof.

LLM-driven code synthesis and evolution. The lineage runs from Codex Chen et al. [2021] and AlphaCode Li et al. [2022] to evolutionary search (FunSearch Romera-Paredes et al. [2023], AlphaDev Mankowitz et al. [2023], AlphaEvolve Balog et al. [2025]). The ADRS programme Cheng et al. [2025] applies the latter via an open-source framework OpenEvolve contributors [2025] across ten systems benchmarks; we narrow it to a single safety-critical function. Agent benchmarks like SWE-bench Jimenez et al. [2024] gate at the repository level rather than the function-and-trace level we operate at.

Deterministic simulation and distributed-systems testing. FoundationDB’s simulator Wilson [2014], Zhou et al. [2021] and Jepsen Kingsbury [2013–present] pioneered seeded simulation and linearizability checking; contemporary continuations include Antithesis Antithesis [2024], TigerBeetle’s VOPR TigerBeetle Engineering [2023–2025], and Madsim Wang [2020]. Property-based testing contributes the underlying seeded-random discipline Claessen and Hughes [2000], MacIver et al. [2019]. Our DST inherits FDB’s single-goroutine seeded-RNG style and the register-history checker pattern of Knossos Kingsbury [2014]/Porcupine Athalye [2017]. The `etcd` project ships a robustness suite targeting the integrated server `etcd` Authors [2022–present]; our harness is the algorithm-library sibling.

9 Conclusion

In a domain where safety properties are mechanically checkable, the harness is the load-bearing artefact. Our pyramid gates an LLM-evolutionary loop over one function in `etcd-io/raft`; the strict gate fires zero false positives across 200 seeds, and the headline accepted candidate delivers a 25.4% paired-median throughput improvement. The methodology generalises to any systems library with a pluggable side-effect interface and a TLA^+ spec.

References

References

Antithesis. The determinator: A deterministic hypervisor for autonomous software testing. <https://antithesis.io/>

355 `//antithesis.com/product/how_antithesis_works/`, 2024.

356 Anish Athalye. Porcupine: A fast linearizability checker in Go. [https://github.com/anishathalye/](https://github.com/anishathalye/porcupine)
357 `porcupine`, 2017.

358 Matej Balog, Alexander Novikov, Ngán Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang,
359 Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Ab-
360 bas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex
361 Davies, Sebastian Nowozin, and Pushmeet Kohli. AlphaEvolve: A Gemini-powered coding
362 agent for designing advanced algorithms. Technical report, Google DeepMind, May 2025.
363 White paper. [https://storage.googleapis.com/deepmind-media/DeepMind.com/Blog/](https://storage.googleapis.com/deepmind-media/DeepMind.com/Blog/alphaevolve-a-gemini-powered-coding-agent-for-designing-advanced-algorithms/AlphaEvolve.pdf)
364 `alphaevolve-a-gemini-powered-coding-agent-for-designing-advanced-algorithms/`
365 `AlphaEvolve.pdf`.

366 Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri
367 Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on
368 code. *arXiv preprint arXiv:2107.03374*, 2021. doi: 10.48550/arXiv.2107.03374.

369 Ming Chen and Sesh Nalla. Closing the verification loop, part 2: Fully autonomous optimization. Datadog Engi-
370 neering Blog, 2026. <https://www.datadoghq.com/blog/ai/fully-autonomous-optimization/>.

371 Audrey Cheng, Shu Liu, Melissa Pan, Zhifei Li, Bowen Wang, Alex Krentsel, Tian Xia, Mert Cemri, Jongseok
372 Park, Shuo Yang, Jeff Chen, Lakshya Agrawal, Aditya Desai, Jiarong Xing, Koushik Sen, Matei Zaharia, and
373 Ion Stoica. Barbarians at the gate: How AI is upending systems research. *arXiv preprint arXiv:2510.06189*,
374 2025. doi: 10.48550/arXiv.2510.06189.

375 Koen Claessen and John Hughes. QuickCheck: A lightweight tool for random testing of Haskell programs.
376 In *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming (ICFP)*,
377 pages 268–279, 2000. doi: 10.1145/351240.351266.

378 etcd Authors. etcd robustness testing framework. [https://github.com/etcd-io/etcd/tree/main/](https://github.com/etcd-io/etcd/tree/main/tests/robustness)
379 `tests/robustness`, 2022–present. Upstream robustness suite validating KV and Watch API guarantees
380 under seeded fault injection.

381 Chris Hawblitzel, Jon Howell, Manos Kapritsos, Jacob R. Lorch, Bryan Parno, Michael Lowell Roberts, Srinath
382 T. V. Setty, and Brian Zill. IronFleet: Proving practical distributed systems correct. In *Proceedings of the 25th*
383 *ACM Symposium on Operating Systems Principles (SOSP '15)*, pages 1–17, 2015. doi: 10.1145/2815400.
384 2815428.

385 Maurice Herlihy and Jeannette M. Wing. Linearizability: A correctness condition for concurrent objects. *ACM*
386 *Transactions on Programming Languages and Systems*, 12(3):463–492, 1990. doi: 10.1145/78969.78972.

387 Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan.
388 SWE-bench: Can language models resolve real-world GitHub issues? In *Proceedings of the International*
389 *Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.06770.

390 Alp Keles, Jai Menon, Sesh Nalla, and Vyom Shah. Closing the verification loop: Observability-driven harnesses
391 for building with agents. Datadog Engineering Blog, 2026. [https://www.datadoghq.com/blog/ai/](https://www.datadoghq.com/blog/ai/harness-first-agents/)
392 `harness-first-agents/`.

393 Kyle Kingsbury. Jepsen: Distributed systems safety analysis. <https://jepsen.io>, 2013–present.

394 Kyle Kingsbury. Knossos: A linearizability checker for distributed systems. [https://github.com/](https://github.com/jepsen-io/knossos)
395 `jepsen-io/knossos`, 2014. Clojure implementation; predecessor to Porcupine.

396 Andrea Lattuada, Travis Hance, Chanhee Cho, Matthias Brun, Isitha Subasinghe, Yi Zhou, Jon Howell, Bryan
397 Parno, and Chris Hawblitzel. Verus: Verifying Rust programs using linear ghost types. *Proceedings of the*
398 *ACM on Programming Languages*, 7(OOPSLA1):286–315, 2023. doi: 10.1145/3586037.

399 Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James
400 Keeling, Felix Gimeno, Agustin Dal Lago, Thomas Hubert, Peter Choy, Cyprien de Masson d’Autume, Igor
401 Babuschkin, Xinyun Chen, Po-Sen Huang, Johannes Welbl, Sven Gowal, Alexey Cherepanov, James Molloy,
402 Daniel J. Mankowitz, Esme Sutherland Robson, Pushmeet Kohli, Nando de Freitas, Koray Kavukcuoglu, and
403 Oriol Vinyals. Competition-level code generation with AlphaCode. *Science*, 378(6624):1092–1097, 2022.
404 doi: 10.1126/science.abq1158.

405 David R. MacIver, Zac Hatfield-Dodds, and Hypothesis contributors. Hypothesis: A new approach to property-
406 based testing. *Journal of Open Source Software*, 4(43):1891, 2019. doi: 10.21105/joss.01891.

- 407 Daniel J. Mankowitz, Andrea Michi, Anton Zhernov, Marco Gelmi, Marco Selvi, Cosmin Paduraru, Edouard
 408 Leurent, Shariq Iqbal, Jean-Baptiste Lesciau, Alex Ahern, Thomas Köppe, Kevin Millikin, Stephen Gaffney,
 409 Sophie Elster, Jackson Broshear, Chris Gamble, Kieran Milan, Robert Tung, Minjae Hwang, A. Taylan
 410 Cemgil, Mohammadamin Barekatain, Yujia Li, Amol Mandhane, Thomas Hubert, Julian Schrittwieser,
 411 Demis Hassabis, Pushmeet Kohli, Martin Riedmiller, Oriol Vinyals, and David Silver. Faster sorting
 412 algorithms discovered using deep reinforcement learning. *Nature*, 618:257–263, 2023. doi: 10.1038/
 413 s41586-023-06004-9.
- 414 Sesh Nalla. redis-rust: A case study in ai-assisted systems programming with deterministic verification. Technical
 415 report, 2026. <https://github.com/nerdsane/redis-rust/blob/main/docs/PAPER.md>.
- 416 Diego Ongaro and John Ousterhout. In search of an understandable consensus algorithm (extended version). In
 417 *Proceedings of the 2014 USENIX Annual Technical Conference (USENIX ATC '14)*, pages 305–319. USENIX
 418 Association, 2014.
- 419 OpenEvolve contributors. OpenEvolve: An open-source evolutionary code-optimization framework. <https://github.com/codelion/openevolve>, 2025.
- 421 Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar,
 422 Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli,
 423 and Alhussein Fawzi. Mathematical discoveries from program search with large language models. *Nature*,
 424 625, 2023. doi: 10.1038/s41586-023-06924-6. Published December 14, 2023.
- 425 TigerBeetle Engineering. VOPR: Viewstamped operation replayer for deterministic simulation testing. <https://github.com/tigerbeetle/tigerbeetle/blob/main/docs/internals/vopr.md>, 2023–2025.
- 427 Runji Wang. Madsim: Magical deterministic simulator for distributed systems. <https://github.com/madsim-rs/madsim>, 2020. Rust async runtime for deterministic simulation testing.
- 429 James R. Wilcox, Doug Woos, Pavel Panchekha, Zachary Tatlock, Xi Wang, Michael D. Ernst, and Thomas
 430 Anderson. Verdi: A framework for implementing and formally verifying distributed systems. In *Proceedings*
 431 *of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '15)*,
 432 pages 357–368, 2015. doi: 10.1145/2737924.2737958.
- 433 Will Wilson. Testing distributed systems with deterministic simulation. Strange
 434 Loop Conference talk, 2014. [https://www.thestrangeloop.com/2014/
 435 testing-distributed-systems-w-slash-deterministic-simulation.html](https://www.thestrangeloop.com/2014/testing-distributed-systems-w-slash-deterministic-simulation.html).
- 436 Jingyu Zhou, Meng Xu, Alexander Shraer, Bala Namasivayam, Alex Miller, Evan Tschannen, Steve Atherton,
 437 Andrew J. Beamon, Rusty Sears, John Leach, Dave Rosenthal, Xin Dong, Will Wilson, Ben Collins, David
 438 Scherer, Alec Grieser, Young Liu, Alvin Moore, Bhaskar Muppana, Xiaoge Su, and Vishesh Yadav. Founda-
 439 tionDB: A distributed unbundled transactional key value store. In *Proceedings of the 2021 International*
 440 *Conference on Management of Data (SIGMOD '21)*, pages 2653–2666, 2021. doi: 10.1145/3448016.3457559.

441 A Technical Appendices and Supplementary Material

Conference For AI Scientists 2026 - AI Involvement Checklist

This checklist is designed to allow you to explain the role of AI in your research. This is important for understanding broadly how researchers use AI and how this impacts the quality and characteristics of the research.

The checklist has two parts: **Research Stage Assessment** (items 1-4), where you rate the level of AI involvement and iteration effort at each stage of the research process, and **AI System Documentation** (items 5-6), where you provide free-text descriptions of the AI systems used and their observed limitations.

Research Stage Assessment

For items 1-4, give a score from the scale below that defines the role of AI in each part of the scientific process. The scores are as follows:

- **[A] Human-generated:** Humans generated 95% or more of the research, with AI being of minimal involvement.
- **[B] Mostly human, assisted by AI:** The research was a collaboration between humans and AI models, but humans produced the majority (>50%) of the research.
- **[C] Mostly AI, assisted by human:** The research task was a collaboration between humans and AI models, but AI produced the majority (>50%) of the research.
- **[D] AI-generated:** AI performed over 95% of the research. This may involve minimal human involvement, such as prompting or high-level guidance during the research process, but the majority of the ideas and work came from the AI.

For each research stage where AI was involved, i.e. where you selected **[B]**, **[C]**, or **[D]**, please also indicate the approximate level of iteration effort required using the provided iteration macros. Count substantive attempts, prompts, runs, agent trajectories, or course corrections; do not count minor wording edits.

- **[Low]** : approximately 1-10 substantive attempts.
- **[Medium]** : approximately 10-100 substantive attempts, with some failed attempts or course corrections.
- **[High]** : approximately 100+ substantive attempts, with substantial exploration, failures, or trial-and-error.
- **[NA]** : not applicable because AI involvement was **[A]**.
- **[Unclear]** : the authors cannot reasonably estimate.

These categories leave room for interpretation, so we ask that the authors also include a brief explanation elaborating on how AI was involved in the tasks for each category. Please keep your explanation to less than 150 words.

1. **Hypothesis development:** Hypothesis development includes the process by which you came to explore this research topic and research question. This can involve the background research performed by either researchers or by AI. This can also involve whether the idea was proposed by researchers or by AI.

Answer: **[TODO]**

Iteration Effort: **[TODO]**

Explanation: **[TODO]**

2. **Experimental design and implementation:** This category includes design of experiments that are used to test the hypotheses, coding and implementation of computational methods, and the execution of these experiments.

Answer: **[TODO]**

Iteration Effort: **[TODO]**

Explanation: **[TODO]**

- 489 3. **Analysis of data and interpretation of results:** This category encompasses any process to
490 organize and process data for the experiments in the paper. It also includes interpretations of
491 the results of the study.
492 Answer: [TODO]
493 Iteration Effort: [TODO]
494 Explanation: [TODO]
- 495 4. **Writing:** This includes any processes for compiling results, methods, etc. into the final
496 paper form. This can involve not only writing of the main text but also figure-making,
497 improving layout of the manuscript, and formulation of narrative.
498 Answer: [TODO]
499 Iteration Effort: [TODO]
500 Explanation: [TODO]

501 AI System Documentation

502 For items 5-6, provide a free-text description.

- 503 5. **AI systems used:** Describe all AI systems used in this research without naming specific
504 products or models. Use system-level descriptors only (e.g., “a frontier large language
505 model”, “a protein structure prediction system”, “a custom multi-agent framework built on
506 open-source models”). Include relevant details such as system type, scale, and capabilities.
507 **Specific model and product names should only be included in the camera-ready version.**
508 Description: A single frontier large language model in agentic coding configuration, accessed
509 through an OpenAI-compatible chat completion endpoint provided by a third-party inference
510 platform. The same model identity was used both as (a) the agentic coding assistant that
511 constructed the verification harness, the evaluator, the regression tooling, and this manuscript,
512 and (b) the inner candidate generator inside the evolutionary optimization loop. An open-
513 source evolutionary code-optimization framework derived from AlphaEvolve provided the
514 MAP-Elites candidate database, parent sampler, and prompt-templating layer; we did not
515 modify the framework. The Porcupine linearizability checker is used as the consistency
516 oracle in the harness but is a deterministic algorithmic component, not an AI system.
- 517 6. **Observed AI Limitations:** What specific limitations and failure modes have you observed
518 when using AI as a partner or lead author?
519 Description: Three failure modes recurred. (1) *Method invention*: the inner candidate
520 generator occasionally proposed code referencing methods that do not exist on the relevant
521 structs (e.g. a hallucinated `IsHealthy` on the progress tracker); these were caught at the
522 compile layer of the harness rather than ending up in deployed code. (2) *Score-metric*
523 *over-fitting*: when the evaluator scored on a min-of-five benchmark, the generator produced
524 candidates whose “speedup” was largely the result of single-run jitter in the lower tail; the
525 apparent fifteen-percent improvement reduced to a paired-median four percent under more
526 rigorous measurement. (3) *Optimistic self-reporting*: when asked to summarise experimental
527 outputs, the agentic coding assistant initially reported the optimistic-end statistic; subsequent
528 prompts requiring paired distributions and explicit noise-floor characterisation produced
529 materially different numbers. We did not measure cross-model stability, and the underlying
530 inference service is stochastic; different runs of the same loop are not expected to produce
531 identical candidates.

532 **Conference For AI Scientists 2026 - Reproducibility and Responsibility**
533 **Checklist**

534 **1. Claims**

535 Question: Do the main claims made in the abstract and introduction accurately reflect the
536 paper's contributions and scope?

537 Answer: [Yes]

538 Justification: The abstract and §1 enumerate four contributions (harness, calibration,
539 evolutionary-search wrapper, empirical study) that map directly to §3, §6.2, §4, and §6.

540 **2. Limitations**

541 Question: Does the paper discuss the limitations of the work performed by the authors?

542 Answer: [Yes]

543 Justification: §7 discusses oracle coverage as a lower bound, the fault catalogue's omissions
544 (crash-restart, slow disk), the synthetic-performance caveat, the harness-vs-deployment
545 distinction, generator stochasticity, and benchmark noise floors.

546 **3. Theory assumptions and proofs**

547 Question: For each theoretical result, does the paper provide the full set of assumptions and
548 a complete (and correct) proof?

549 Answer: [NA]

550 Justification: The paper does not include theoretical results. The Raft invariants we check
551 come from prior published work Ongaro and Ousterhout [2014] and the in-tree TLA⁺
552 specification; we do not extend or re-prove them.

553 **4. Experimental result reproducibility**

554 Question: Does the paper fully disclose all the information needed to reproduce the main ex-
555 perimental results of the paper to the extent that it affects the main claims and/or conclusions
556 of the paper (regardless of whether the code and data are provided or not)?

557 Answer: [Yes]

558 Justification: §5 records the pinned upstream commit (etcd-io/raft @ 67d129e8, 2026-
559 05-26), the Go and tooling versions (Go 1.26.3, OpenEvolve 0.2.27, Porcupine v1.1.0,
560 OpenJDK 17), the hardware configuration (dedicated 16-vCPU x86_64 cloud host, 32 GB,
561 Ubuntu 24.04), the seeded fault parameters, the seed set, and the benchmark methodology
562 (Go's standard `go test -bench=... -benchtime=1s -count=5` with min-reduction).
563 The single upstream patch enabling deterministic seeded election timeouts is described in
564 §3.

565 **5. Open access to data and code**

566 Question: Does the paper provide open access to the data and code, with sufficient instruc-
567 tions to faithfully reproduce the main experimental results, as described in supplemental
568 material?

569 Answer: [Yes]

570 Justification: The harness source, evaluator, OpenEvolve configuration, prompt templates,
571 and SQLite candidate database will be released as the verification artefact on acceptance,
572 alongside the per-seed reproduction scripts referenced in §6. The release will be anonymised
573 at submission and de-anonymised at camera-ready per CAISc policy.

574 **6. Experimental setting/details**

575 Question: Does the paper specify all the training and test details (e.g., data splits, hyper-
576 parameters, how they were chosen, type of optimizer, etc.) necessary to understand the
577 results?

578 Answer: [Yes]

579 Justification: §5 and §4 detail cluster size, tick budget, fault probabilities, scenario pa-
580 rameters, search space, generator temperature, max-tokens, and evaluator timeouts. The
581 OpenEvolve config YAML is included in the artefact.

582 **7. Experiment statistical significance**

583 Question: Does the paper report error bars suitably and correctly defined or other appropriate
584 information about the statistical significance of the experiments?

585 Answer: [Yes]

586 Justification: §6.3 (Table 1) reports the min, median, and max of a paired ten-run benchmark
587 distribution rather than single-point speedups, and §7 explicitly characterises the bimodal
588 benchmark distribution and a ~10% noise-floor below which apparent improvements are
589 not interpretable as real. We do not perform parametric hypothesis testing because the
590 underlying distribution violates normality.

591 **8. Experiments compute resources**

592 Question: For each experiment, does the paper provide sufficient information on the com-
593 puter resources (type of compute workers, memory, time of execution) needed to reproduce
594 the experiments?

595 Answer: [Yes]

596 Justification: §5 states the single-host commodity workstation context and the
597 embarrassingly-parallel-by-seed nature of the harness. §6.5 reports the per-iteration ~57 s
598 wall-clock decomposition, the ~76-minute total run time, and the approximate ten-dollar
599 inference spend.

600 **9. Research Ethics**

601 Question: Does the research conducted in the paper conform with the NeurIPS Code of
602 Ethics, except where CAISc’s AI authorship policies apply?

603 Answer: [Yes]

604 Justification: The research uses no human subjects, no personally identifiable data, and no
605 proprietary corpora. The AI Involvement Checklist documents the role of the language
606 model. The work targets a published open-source library; we do not introduce undisclosed
607 modifications and we propose no production deployment.

608 **10. Broader impacts**

609 Question: Does the paper discuss both potential positive societal impacts and negative
610 societal impacts of the work performed?

611 Answer: [Yes]

612 Justification: Positive: harness-gated agentic loops lower the cost of safety-preserving
613 optimisation in safety-critical infrastructure, and the verification-pyramid pattern generalises
614 to other distributed-systems libraries with published TLA⁺ specifications. Negative: the
615 same loop could be misused to produce subtly-incorrect code that nonetheless clears a
616 weak harness; we mitigate by emphasising in §7 that harness-acceptance is not deployment-
617 readiness and that the oracle’s coverage is a lower bound on bug-finding capability.